Implementation of a Configurable PWM Generator

Computer Architecture Project

Team: Matei David, Atudosiei Andrei Cristian, Vrabie Tudor

Contents

1 Introduction

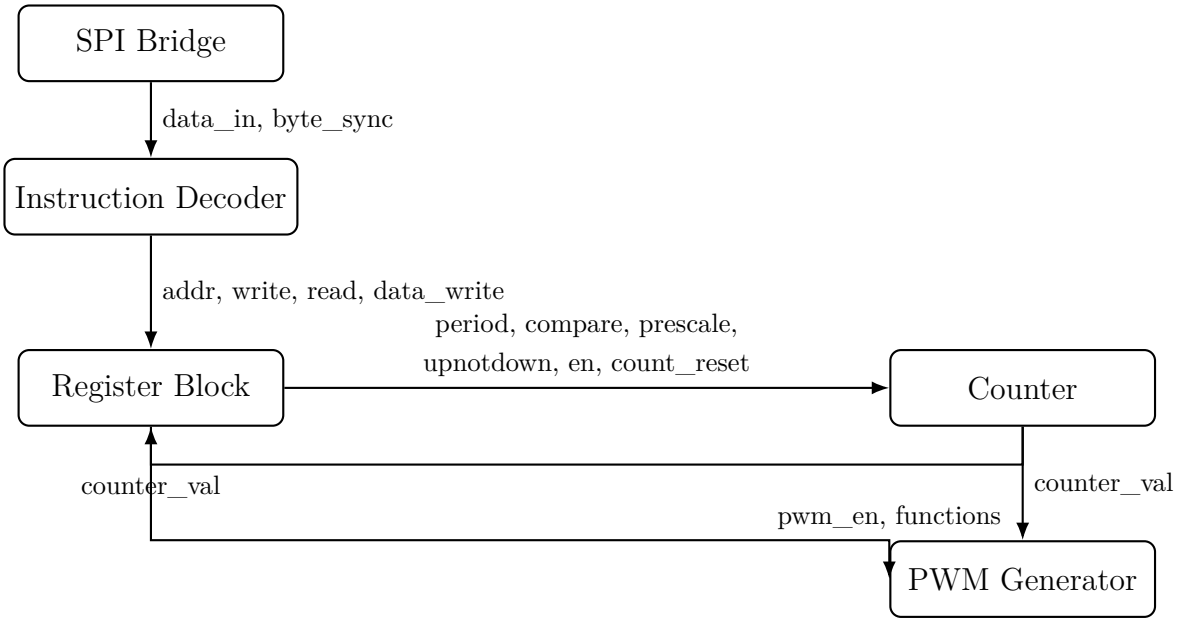
PWM (Pulse Width Modulation) signal generators are essential components in many embedded systems, being used for precise control of devices such as LEDs, electric motors, and power electronics. In modern microcontroller architectures, these generators are typically implemented as programmable peripherals, configurable through internal registers and serial communication interfaces.

In this project, we implemented a complete hardware module capable of generating a configurable PWM signal using the Verilog hardware description language. The peripheral allows the configuration of the main PWM parameters, such as period, duty cycle, alignment mode, and counting direction through an SPI communication interface, as well as the activation, deactivation, and monitoring of the internal counter.

The architecture of the module consists of five main components: the SPI bridge responsible for receiving and transmitting data, the instruction decoder which interprets byte-level commands, the register block which stores the peripheral configuration, the `counter` module which performs the actual counting operation, and the `pwm_gen` module which generates the PWM output signal. The implementation of each component, along with the specific design decisions taken throughout the project, is presented in the following sections.

2 Architecture Overview

The PWM generator peripheral was designed as a modular hardware system composed of five interconnected components, each responsible for a distinct stage of data processing and signal generation. The architecture follows a clear separation of responsibilities, ensuring synchronous operation, predictable data flow, and ease of verification.



2.1 Module Interactions

The communication flow across components follows a strict pipeline:

1. **SPI Bridge** receives serial data from the external master and converts it into 8-bit parallel words. It also asserts a `byte_sync` signal to indicate the completion of each byte transfer.

2. **Instruction Decoder** interprets these bytes as structured commands. Based on the first byte, it determines whether the operation is a read or write, identifies the target register, and decides whether the high or low byte of a 16-bit register is being accessed.

3. **Register Block** stores all configuration parameters of the peripheral: period, compare values, prescaler settings, enable bits, and internal modes. It exposes these parameters continuously to the counter and PWM generator.

4. **Counter Module** implements the time base of the PWM generator. It increments or decrements based on the configuration, handles prescaling, detects overflow and underflow, and outputs the current counter value.

5. **PWM Generator** produces the final PWM output based on the counter value and compare thresholds. It supports left-aligned, right-aligned, and dual-compare modes, making the output waveform fully configurable.

2.2 Signal Flow Between Components

The following summarizes the key signals exchanged among modules:

- **From SPI Bridge -> Instruction Decoder:**
  - `data_in[7:0]`: parallel byte received from MOSI
  - `byte_sync`: indicates end of an 8-bit transfer
- **From Instruction Decoder -> Register Block:**
  - `addr[5:0]`: target register address
  - `write`, `read`: operation type
  - `data_write[7:0]`: byte to be written into a register
- **From Register Block -> Counter:**
  - `period`, `compare1`, `compare2`
  - `prescale`, `upnotdown`, `counter_reset`, `counter_en`

- **From Counter -> PWM Generator:**
  - `counter_val`: current counter state
  - overflow/underflow signals
- **From Register Block -> PWM Generator:**
  - `functions`: waveform mode configuration
  - `pwm_en`: enables PWM generation

2.3 Design Considerations

Several design choices were made to ensure a clean, predictable, and hardware-accurate implementation of the PWM peripheral. The decisions below reflect the actual behaviour of the modules implemented in Verilog:

- **Single clock domain:** All modules operate on the same peripheral clock `clk`. This avoids metastability issues, removes the need for synchronization FIFOs, and ensures that all state transitions are fully deterministic.
- **Clear separation between communication logic and functional logic:** The `spi_bridge` and `instr_dcd` modules handle all SPI-related behaviour (bit shifting, byte alignment, decoding of commands). The functional modules (`regs`, `counter`, `pwm_gen`) remain purely synchronous blocks that never depend on SPI timing. This simplifies the design and allows each block to be verified independently.
- **Direct register-to-datapath connection:** All configuration values (period, prescaler, compare values, mode bits) are stored in the register block and continuously forwarded to the counter and PWM generator. Updates take effect immediately in the next clock cycle, exactly matching the behaviour of simple microcontroller peripherals.
- **Pulse-based reset mechanism for the counter:** The `COUNT_RESET` write operation generates a two-cycle pulse using an internal countdown register. This guarantees a clean synchronous reset without requiring edge detection or asynchronous logic.
- **Prescaler implemented using a power-of-two divider:** Instead of a configurable comparator, the prescaler computes its limit as $(2^{prescale} - 1)$. This keeps the hardware minimal and efficient while still providing a wide dynamic range.
- **Fully combinational PWM logic with synchronous output register:** The PWM waveform is computed combinationally from the counter value and mode bits,

while the final output is registered. This ensures glitch-free transitions and stable timing on the output pin.

- **Minimal hardware footprint and ease of extensibility:** The register map is simple and contiguous, and additional PWM channels or features can be added without restructuring existing logic.

These considerations lead to a modular, predictable, and easily verifiable hardware design that closely follows the structure of real PWM peripherals found in microcontrollers.

3 SPI Bridge

The `spi_bridge` module is responsible for interfacing the external SPI master with the internal parallel datapath of the PWM peripheral. Its role is to convert the serial MOSI bitstream into 8-bit parallel words and to serialize the outgoing data on the MISO line according to standard SPI timing. This component ensures protocol correctness, byte alignment, and data synchronization for all SPI transactions.

3.1 Functional Description

The module monitors the `sclk` and `cs_n` signals coming from the SPI master. When the chip select is asserted (active low), the bridge begins sampling MOSI on every rising edge of the peripheral clock and shifts the received bits into an 8-bit shift register. A 3-bit counter tracks the number of bits received.

Once eight bits have been collected, the bridge assembles them into a full byte and asserts the `byte_sync` signal for exactly one clock cycle. This signal notifies the instruction decoder that a new valid byte is available on the `data_in` bus.

Outgoing data is handled symmetrically. The bridge takes an 8-bit value provided on the internal `data_out` bus and outputs its bits sequentially on the MISO line. The transmitted bit is selected based on the current value of the bit counter, ensuring that the MSB is transmitted first.

3.2 Internal Signals and Registers

The SPI bridge uses several internal registers to track its state:

- **shift_reg** – stores the MOSI bits as they arrive, forming the incoming byte.
- **bit_cnt** – a 3-bit counter that increments for each incoming bit and resets after 7.
- **data_in_r** – the completed received byte exposed to the internal bus.
- **byte_sync_r** – a one-cycle pulse indicating that a new byte is ready.
- **miso_r** – the registered output bit for the MISO line.

All internal logic operates synchronously with the system clock `clk`, while respecting the state of the external SPI control signals. When `cs_n` is deasserted, the module resets its counters and ignores incoming transitions, ensuring clean transaction boundaries.

3.3 Data Flow

The operational data flow can be summarized as follows:

1. When `cs_n` is driven low, the module becomes active and begins shifting MOSI bits into `shift_reg`.
2. For each received bit, the bridge simultaneously outputs the corresponding bit of `data_out` on MISO.
3. After eight bits, the fully assembled byte is stored in `data_in_r`, and `byte_sync_r` is asserted.
4. Transaction ends when `cs_n` returns high, causing the bit counter to reset.

This behavior ensures that communication remains byte-aligned and deterministic, which is essential for the instruction decoder and register block that depend on reliably synchronized control data.

3.4 Design Considerations

Several design choices were made for robustness:

- **Synchronous design:** all internal flip-flops use the system clock instead of `sclk`, making timing easier to analyze and preventing metastability.
- **Atomic byte reception:** `byte_sync` is asserted only when a full 8-bit word is available, eliminating partial transfers.
- **MSB-first serialization:** outgoing data always respects the standard SPI bit order.
- **Graceful idle handling:** when `cs_n` is high, all internal counters reset, ensuring clean transaction boundaries.

Overall, the `spi_bridge` provides a clean and reliable SPI-to-parallel interface, serving as the foundation for all configuration operations within the PWM peripheral.

4 Instruction Decoder

The `instr_dcd` module acts as the central control element of the peripheral, decoding the byte-level protocol received from the SPI bridge and converting it into precise read or write operations on the register block. Its role is to interpret each pair of bytes transmitted over SPI according to the peripheral's custom instruction format.

The decoder operates as a two-phase finite-state machine (FSM) synchronized with the `byte_sync` signal generated by the SPI interface. This structure ensures that every command is processed atomically and prevents partial or inconsistent register accesses.

4.1 Instruction Format

Each SPI transaction targeting this peripheral consists of two bytes:

- **Setup Byte:**
  - Bit 7: Read/Write flag (1 = write, 0 = read)
  - Bit 6: High/Low byte selector
  - Bits 5:0: Register address
- **Data Byte:**
  - Payload byte to be written (for write operations)
  - Ignored for read operations (the response is generated immediately)

This compact format guarantees a minimal instruction width while covering all configuration registers in the system.

4.2 FSM Operation

The module consists of a single flip-flop FSM represented by the `phase` signal:

- **Phase 0 — Setup Phase:**
  The decoder interprets the first byte of the command and stores:
  - the operation type (`rw_reg`)
  - the high/low selector (`hl_reg`)
  - the register address (`addr_reg`)
- **Phase 1 — Data Phase:**
  On the next `byte_sync`, the decoder finalizes the operation:

- For **write** commands: `write` is asserted, and `data_write` is forwarded to the register block.
- For **read** commands: `read` is asserted, and `data_out` returns the value from `data_read`.

After completing the operation, the FSM returns to phase 0.

4.3 Output Signals

The module exposes the following control signals to the register block:

- `read`: one-cycle pulse indicating a register read
- `write`: one-cycle pulse indicating a register write
- `addr[5:0]`: register address
- `data_write[7:0]`: byte to be written
- `data_out[7:0]`: value returned during a read

Only one of `read` or `write` is asserted in a given cycle, ensuring deterministic register access.

4.4 Design Considerations

Several design decisions were made to ensure reliability:

- **Byte-synchronized decoding**: All operations are gated by `byte_sync`, ensuring alignment with SPI transfers.
- **Two-phase FSM**: Simplifies timing closure and avoids partial or ambiguous commands.
- **Address latching**: The address is captured during the setup phase to prevent mismatched accesses.
- **Immediate read response**: Read operations propagate `data_read` directly through `data_out` with minimal latency.

This architecture results in a compact, deterministic, and fully synchronous instruction decoder, closely resembling the control logic found in real microcontroller peripherals.

5 Register Block

The `regs` module implements the internal register file of the PWM peripheral. It provides storage for all configuration parameters used by the counter and PWM generator, while also exposing read and write access to the instruction decoder through a simple address–data interface. The block acts as the central configuration memory of the system, ensuring deterministic and synchronous updates for all operational parameters.

5.1 Register Organization

The register map is structured into 8-bit addressable entries, with multi-byte fields (e.g., 16-bit period or compare values) split across consecutive addresses. Each register can be accessed through the standard interface:

- `read`, `write`: one-cycle control pulses from the decoder
- `addr[5:0]`: 6-bit register address
- `data_write[7:0]`: incoming byte for write operations
- `data_read[7:0]`: outgoing byte for read operations

Internally, the block stores the following configuration values:

- **Period register** (`period_reg`, 16-bit)
- **Enable bit** (`en_reg`)
- **Compare 1 register** (`compare1_reg`, 16-bit)
- **Compare 2 register** (`compare2_reg`, 16-bit)
- **Counter reset pulse register** (`count_reset_reg`)
- **Counting direction bit** (`upnotdown_reg`)
- **Prescaler register** (`prescale_reg`, 8-bit)
- **PWM enable bit** (`pwm_en_reg`)
- **Functions register** (`functions_reg`, 8-bit)

These values are continuously exposed to the `counter` and `pwm_gen` modules.

5.2 Read Logic

The read path is implemented as a combinational multiplexer that selects the appropriate byte based on the incoming address. The decoder issues a `read` pulse, and the module returns the requested value through `data_read` during the same cycle.

The following information can be read:

- low and high bytes of `period` (addresses `0x00`–`0x01`)
- enable bit (`0x02`)
- `compare1` and `compare2` registers (`0x03`–`0x06`)
- current counter value, forwarded from the `counter` module (`0x08`–`0x09`)
- prescaler value (`0x0A`)
- direction bit (`0x0B`)
- PWM enable (`0x0C`)
- waveform configuration byte (`0x0D`)

All other addresses return zero.

5.3 Write Logic

Write operations take effect synchronously on the rising edge of `clk`. When the decoder asserts the `write` signal, the module stores the incoming byte into the corresponding internal register:

- addresses `0x00`–`0x01`: update the 16-bit period value
- `0x02`: update the enable bit
- `0x03`–`0x04`: update `compare1`
- `0x05`–`0x06`: update `compare2`
- `0x07`: initiate a counter reset pulse
- `0x0A`: update prescaler
- `0x0B`: update direction bit
- `0x0C`: update PWM enable
- `0x0D`: update functions register

5.4 Counter Reset Pulse Generation

The peripheral must generate a clean, short pulse to reset the counter without causing metastability. To achieve this, the `count_reset_ctr` register implements a two-cycle pulse generator:

- writing to address `0x07` loads the counter with value `2`
- as long as the counter is non-zero, `count_reset` is asserted
- the counter decrements each clock cycle until it reaches zero

This technique guarantees a deterministic and glitch-free reset event.

5.5 Design Considerations

Several design aspects contribute to the reliability of the register block:

- **Strict separation of read and write paths**, preventing unintended overlap.
- **Combinational read logic** ensures immediate access latency.
- **Synchronous writes** avoid metastability and ensure correct timing.
- **Safe handling of counter reset** using a dedicated pulse generator.
- **Modular register map** that can be easily extended in future revisions.

Overall, the register block serves as the configuration backbone of the PWM peripheral, providing stable storage and reliable updates for all operational parameters.

6 Counter Module

The `counter` module provides the time base for the PWM generator by implementing a configurable 16-bit up/down counter with an integrated prescaler. Its purpose is to divide the peripheral clock according to the user-programmed prescale value and to generate periodic counter increments or decrements depending on the selected mode.

The counter operates entirely synchronously with the main peripheral clock and supports three core functionalities: prescaling, directional counting, and overflow/underflow–based wrap-around.

6.1 Functional Description

The counter consists of two internal blocks:

- **Prescaler Block** – divides the input clock according to the value stored in the `PRESCALE` register. The prescaler produces a `tick_enable` pulse whenever it reaches its limit, signalling that a new counter step may occur.
- **Counter Block** – updates the internal 16-bit counter on each prescaler tick, depending on the selected counting direction.

A simplified view of its behaviour:

1. The prescaler increments each clock cycle while the counter is enabled.
2. Once it reaches the value $(2^{prescale} - 1)$, it resets to zero and asserts `tick_enable`.
3. If `tick_enable = 1` and the counter is enabled:
    - In **up-counting mode**, the counter increments until it reaches `PERIOD`, then wraps to 0.
    - In **down-counting mode**, the counter decrements until it reaches 0, then wraps to `PERIOD`.
4. Both the prescaler and counter reset immediately when the `COUNT_RESET` signal is asserted.

6.2 Prescaler Implementation

The prescaler uses an internal 16-bit register, `prescale_count`, which accumulates clock cycles. The upper bound of this counter is dynamically computed as:

$$prescale\_limit = (1 \ll prescale) - 1$$

Once `prescale_count` reaches `prescale_limit`, it triggers a counter step and resets. This allows exponential scaling of the counting frequency, identical to the prescalers used in many commercial PWM peripherals.

6.3 Counter Behaviour

The counter's internal value is stored in `internal_count`. It is updated only when both:

- the counter is enabled (`EN = 1`),
- and the prescaler indicates a tick.

Depending on the control bit `UPNOTDOWN`, the counter supports two modes:

- **Up-counting**: count from 0 to `PERIOD`, then wrap to 0.
- **Down-counting**: count from `PERIOD` down to 0, then wrap back.

This wrap-around behaviour is essential for generating stable, continuous PWM cycles.

6.4 Reset Logic

Both the prescaler and internal counter implement synchronous reset logic driven by the `COUNT_RESET` register. When activated:

- The prescaler immediately resets to 0.
- The internal counter resets to 0.

This ensures a clean restart of the PWM period without producing invalid edges or glitches.

6.5 Interface Signals

- **count_val[15:0]** – current counter value (read by the register block)
- **period[15:0]** – user-configured terminal value
- **en** – enables or disables counting
- **count_reset** – forces synchronous reset
- **upnotdown** – selects between increment or decrement mode
- **prescale[7:0]** – determines prescaler exponent

6.6 Design Considerations

- The exponential prescaler was chosen to match typical microcontroller PWM implementations, providing a wide dynamic range with minimal hardware.
- Both counter direction and period are fully runtime-configurable.
- The counter emits no direct overflow flag; instead, the PWM generator uses the wrap-around behaviour to detect period boundaries.
- The design guarantees glitch-free operation by gating all updates on clock edges and prescaler ticks.

7 PWM Generator Module

The `pwm_gen` module produces the final PWM output signal based on the counter value and the waveform configuration programmed through the register block. It implements three output modes: left-aligned PWM, right-aligned PWM, and window (dual-compare) PWM. The module ensures synchronous and glitch-free updates to the output signal.

7.1 Functional Description

The PWM generator receives several configuration parameters:

- **pwm_en** – enables or disables the PWM output
- **functions** – selects the PWM mode and alignment
- **compare1, compare2** – define PWM thresholds
- **count_val** – real-time counter value from the `counter` module
- **period** – full PWM cycle duration

Based on these signals, the module determines whether the output should be logic high or low at any given clock cycle. A combinational block computes the desired PWM state, and a synchronous block registers the value at the output to ensure stability and proper timing.

7.2 PWM Modes

The behaviour of the module is determined by two bits in the `functions` register:

$$functions[1:0] = \begin{cases} 00 & \text{Left-aligned PWM} \\ 01 & \text{Right-aligned PWM} \\ 1x & \text{Window (dual-compare) PWM} \end{cases}$$

The three modes operate as follows:

- **Left-aligned PWM (functions[1]=0, functions[0]=0):** The signal starts high at the beginning of the period and switches low once the counter reaches `compare1`.

$$pwm = (count\_val < compare1)$$

- **Right-aligned PWM (functions[1]=0, functions[0]=1):** The logic is inverted, producing a waveform mirrored within the period.

$$pwm = (count\_val < compare1) \ ? \ 0 : 1$$

- **Window / Unaligned PWM (functions[1]=1):** PWM is high only when the counter lies in the interval:

$$compare1 \leq count\_val < compare2$$

  This mode supports arbitrary active windows and is typically used for advanced waveform shaping.

7.3 Combinational Output Logic

The combinational block computes `pwm_logic_out` solely from counter and compare values, without any state. This ensures that all PWM transitions occur immediately when counter thresholds are crossed.

- If window mode is selected, `compare1` and `compare2` form a high-level interval.
- Otherwise, the alignment bit determines whether the pulse is left- or right-aligned.

This design keeps the logic simple and prevents dependencies on previous cycles.

7.4 Synchronous Output Register

The output is registered on the rising edge of `clk`. This register stage provides:

- glitch-free transitions,
- clean timing with respect to the rest of the peripheral,
- a well-defined behaviour when `pwm_en` is disabled.

If `pwm_en = 0`, the output is forced to logic `0`, ensuring deterministic behaviour in idle mode.

7.5 Design Considerations

- A combinational-to-registered architecture was selected to match typical hardware PWM peripherals found in microcontrollers.
- Window mode allows more complex waveforms without needing an additional counter or state machine.
- All comparisons are made using the current counter value, ensuring precise edge placements.
- The design is fully synchronous, operating inside the same clock domain as the counter.

8 Conclusions

The project successfully delivers a complete and fully functional hardware implementation of a configurable PWM generator, closely resembling the peripheral structures used in real microcontroller architectures. By dividing the design into independent modules: SPI bridge, instruction decoder, register block, counter, and PWM generator; the system achieves a clear separation of responsibilities and a clean communication flow.

Throughout the development process, we ensured full synchronous operation, glitch-free output behaviour, and runtime configurability of all parameters, including period, duty cycle, waveform alignment, counting direction, and prescaler settings. The interaction between modules was carefully designed to avoid hazards, ensure updates, and maintain deterministic timing behaviour.

The resulting PWM peripheral is modular, easily extendable, and straightforward to integrate in larger digital systems. Its structure makes it suitable both for educational purposes and for use as a building block in more complex embedded architectures.

9 Bibliography

References

[1] https://cs-pub-ro.github.io/computer-architecture/Evaluare/Teme/PWMGen%20AA%202025/